



IIC1001 — Algoritmos y Sistemas Computacionales — 2026-1

Actividad Evaluada 6 - Complejidad asintótica

Jueves 11-Junio-2026

Instrucciones

Deberán realizar esta actividad dentro de sus grupos designados según Canvas.

Entrega: Jueves 11-Junio, 12:10 mediante foto en Canvas

- Pueden usar apuntes durante la actividad.
- Pueden hacer consultas al equipo docente durante la actividad.
- Deben resolver manualmente, sin ayuda de computador u otros dispositivos electrónicos.
- No se realiza en computador.
- Cualquier miembro puede hacer la entrega.

Evaluación de complejidad en códigos [10 pts]

Luego de un arduo trabajo con Mettaton, decides volver a tu casa, logras entrar, te acuestas en tu cama a descansar y justo cuando estabas por tomar una siesta (ignorando completamente las obligaciones académicas existentes), tu celular vibra, ¡es un mensaje de ayuda de **Cabra!**, parece que aún no es momento del break.

Tu buen ayudante vecino Cabra te comenta que decidió ayudar al profesor (y actual Jefe de carrera) **Yadran Eterovic** con el curso de Estructuras de datos y algoritmos, pero no esperaba que le fueran a asignar la tarea de asignar la complejidad de cada algoritmo propuesto por el profesor.

Identifica la complejidad asintótica de cada uno de los algoritmos a continuación y justifica el procedimiento:

Ejemplo del procedimiento: Supongamos que tenemos el siguiente algoritmo (y suponemos que print toma $\mathcal{O}(1)$):

```
def PrintCount(n):  
    for i in range(1, n + 1):  
        print(i)
```

R: La complejidad asintótica del código (en términos de tiempo de ejecución) es $\mathcal{O}(n)$, ya que, notamos que el algoritmo cuenta únicamente con un ciclo que depende del valor de entrada n , además **print** tiene complejidad $\mathcal{O}(1)$, por lo que, el algoritmo ejecutará n veces ese algoritmo de complejidad constante, dejándonos notación $\mathcal{O}(n \cdot 1) = \mathcal{O}(n)$.

En base al ejemplo anterior, describa la complejidad asintótica de los siguientes algoritmos y justifique su respuesta. Puede justificarse con algún procedimiento matemático o con palabras. (Como pequeño Hint, generalmente es bueno entender qué hace el algoritmo exactamente, y qué secciones dependen de algún parámetro de entrada).

1.- Algoritmo de suma [2 pt]

```
def SumaAcumulativa(n):  
    num = 1  
    for i in range(n):  
        num = num + i  
    return num
```

2.- Algoritmo de Conteo [2 pts]

```
def DobleCount(n, m):  
    for j in range(n):  
        print(j)  
    for k in range(m):  
        print(k)
```

3.- Algoritmo de Gigaconteo [2 pts]

```
def CountGigante(n, m):  
    for i in range(3000):  
        for j in range(n):  
            for k in range(m):  
                print(i)  
                print(j)  
                print(k)
```

4.- Algoritmo de ordenamiento [2 pts]

```
def Insercion(A, n):  
    # n es el largo del array A, equivalente a decir len(A)  
    for i in range(1, n):  
        key = A[i]  
        j = i - 1  
        while j >= 0 and A[j] > key:  
            A[j + 1] = A[j]  
            j = j - 1  
        A[j + 1] = key  
    return A
```

5.- Algoritmo de detección [2 pts]

```
def EsPrimo(n):  
    if n <= 1:  
        return False  
    i = 2  
    while i * i <= n:  
        if n % i == 0:  
            return False  
        i = i + 1  
  
    return True
```
